

Programação

Dispositivos Móveis

aula 14

- No Android, existe uma tarefa (“thread”) principal que é responsável por desenhar a tela e lidar com os eventos de toque na tela.
- Esta “thread” é conhecida como “UI thread” (User Interface Thread), ou também como “main thread”.
- Se o desenvolvedor não usar nenhum tipo de concorrência, todo o código irá rodar nesta “thread” principal.
- Isso se torna um problema para tarefas que levam muito tempo a serem executadas, pois enquanto a tarefa está sendo executada, a interface **não responderá** a eventos, como toques feito pelo usuário.

- Se houver qualquer processamento que ocupe a UI thread por mais de 5 segundos, a aplicação irá receber automaticamente um ANR (Application not responding), e o sistema irá fechar a aplicação.
- Por isso, qualquer processamento mais lento deve ser feito em outras “threads” para não ocupar a UI thread.

- No Android é suportado o mesmo tipo de concorrência dos demais aplicativos Java.
- É possível utilizar “threads”, que executam objetos do tipo “Runnable”.
- Entretanto não podemos alterar nada relativo a UI dentro destas “threads” que rodam em background.
- Apenas a UI thread é que pode alterar a interface de usuário.

- Para contornar esse problema utiliza-se "Handlers".
- "Handler" é um objeto que possui o método "post (Runnable)".
- O objeto "Runnable" que é passado ao método "post" é executado posteriormente dentro da "main thread" e por isso pode realizar alterações na interface da aplicação.
- Outra alternativa que não envolve criar um "Handler" é utilizar o método "runOnUiThread(Runnable)", que pertence à Activity.
- O objeto "Runnable" que é passado a este método também é executado dentro da "main thread".

- Para definir as tarefas que uma aplicação Java deve executar, deve-se criar classes que implementam a interface "Runnable".
- Essa interface possui apenas o método "run()".
- O método run() é conceitualmente análogo ao método main() pois o primeiro funciona como "ponto de partida" de uma tarefa de uma aplicação e o segundo funciona como "ponto de partida" de uma aplicação.

```
class TarefaImprimeOi implements Runnable {  
    public void run() {  
        for(int i = 0; i < 100; i++) {  
            System.out.println("OI");  
        }  
    }  
}
```

- As tarefas são executadas “dentro” de objetos da classe “Thread”.
- Para cada tarefa que se deseja executar, é necessário criar um objeto da classe “Thread” e associar ao objeto que define a tarefa.
- Depois de associar uma tarefa (objeto de uma classe que implementa “Runnable”) a um objeto da classe Thread, devemos “disparar” a execução da “thread” através do método start().

```
class TarefaImprimeOi implements Runnable {  
    public void run() {  
        for(int i = 0; i < 100; i++) {  
            System.out.println("OI");  
        }  
    }  
}
```

```
TarefaImprimeOi tarefa = new TarefaImprimeOi();  
Thread thread = new Thread(tarefa);  
thread.start();
```

Controlando a execução de Threads

- Controlar a execução das tarefas de uma aplicação envolve, por exemplo, decidir quando uma tarefa pode executar, quando não pode, a ordem na qual duas ou mais tarefas devem ser executadas, etc.
- A classe Thread oferece alguns métodos para controlar a execução das tarefas de uma aplicação.

sleep()

- Durante a execução de uma thread, se o método sleep() for chamado a thread ficará sem executar pelo menos durante a quantidade de tempo passada como parâmetro para este método.

```
// Faz a thread corrente dormir por 3 segundos  
Thread.sleep(3000);
```

InterruptedException

- Uma thread que está “dormindo” pode ser interrompida por outra thread.
- Quando isso ocorrer, a thread que está “dormindo” recebe uma “InterruptedException”.

```
try {  
    Thread.sleep(3000);  
} catch (InterruptedException e) {  
  
}
```

Criar um projeto de acordo com os seguintes dados abaixo:

- Application Name: Threads
- Company Domain : app.usuario
- Project location : (Fica a sua escolha)
- Activity Name: MainActivity
- Layout Name : activity_main
- Title : Exemplo com threads

Layout "activity_main.xml"

```
<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:id="@+id/textView"
        android:text="Barra de progresso:" />
    <ProgressBar
        android:id="@+id/progress_bar"
        style="?android:attr/progressBarStyleHorizontal"
        android:max="10" />
    <Button
        android:id="@+id/start_button"
        android:text=" @string/start"/>

</android.support.constraint.ConstraintLayout>
```

Arquivo “res/values/strings.xml”

activity_main.xml ×
MainActivity.java ×
strings.xml ×

Edit translations for all locales in the translations editor.

```

1  <resources>
2      <string name="app_name">Threads</string>
3      <string name="menu_settings">Settings </string >
4      <string name="title_activity_main"> MainActivity </string >
5      <string name="start">Iniciar </string >
6  </resources>

```

Arquivo "MainActivity.java"

```

activity_main.xml x MainActivity.java x strings.xml x
1 package com.example.a364671.threads;
2
3 import android.os.Handler;
4 import android.support.v7.app.AppCompatActivity;
5 import android.os.Bundle;
6 import android.view.View;
7 import android.widget.Button;
8 import android.widget.ProgressBar;
9
10 public class MainActivity extends AppCompatActivity {
11
12     private Handler handler ;
13     private ProgressBar progress ;
14     private Button startButton ;
15
16     @Override
17     protected void onCreate(Bundle savedInstanceState) {
18         super.onCreate(savedInstanceState);
19         setContentView(R.layout.activity_main);
20         progress = (ProgressBar) findViewById (R.id.progress_bar);
21         startButton = (Button) findViewById (R.id.start_button);
22         handler = new Handler();
23         startButton.setOnClickListener (new View.OnClickListener() {

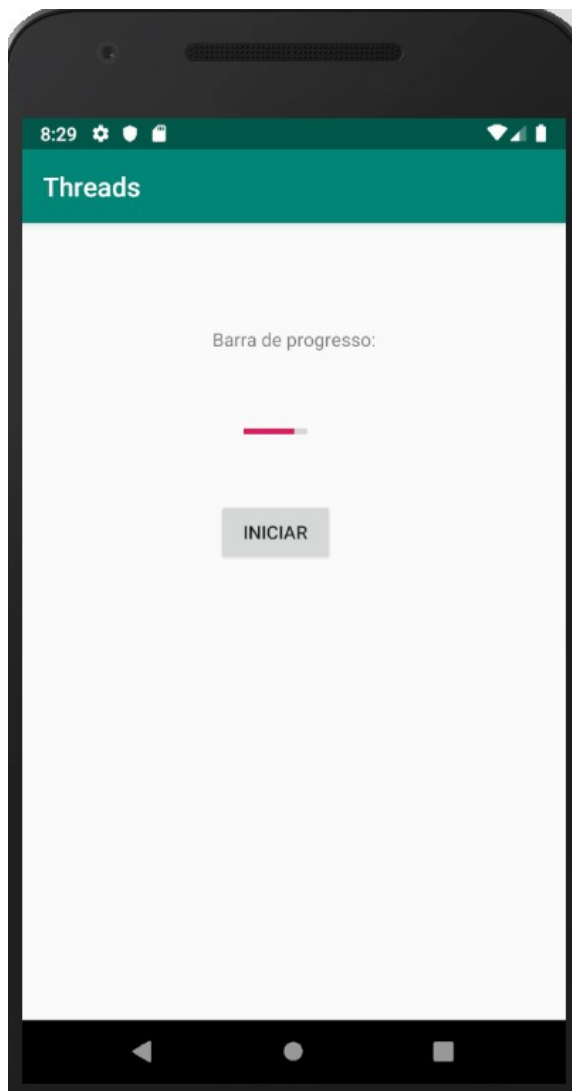
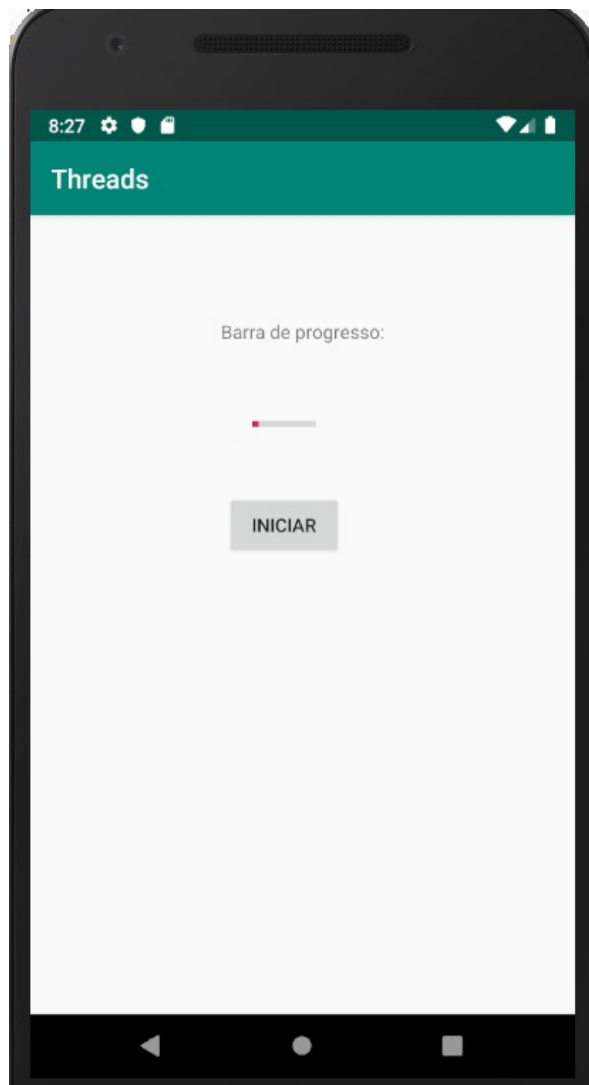
```

Arquivo "MainActivity.java"

```

23     startButton.setOnClickListener (new View.OnClickListener() {
24         @Override
25         public void onClick(View v) {
26             Runnable runnable = new Runnable() {
27                 @Override
28                 public void run() {
29                     for (int i = 1; i <= 10; i++) {
30                         final int value = i;
31                         try {
32                             Thread.sleep( millis: 1000) ;
33                         } catch (InterruptedException e) {
34                             e.printStackTrace();
35                         }
36                         handler.post(new Runnable() {
37                             @Override
38                             public void run() {
39                                 progress.setProgress(value);
40                             }
41                         });
42                     }
43                 }
44             };
45             new Thread(runnable).start();
46         }
47     });
48 }
49

```



Alteração "MainActivity.java"

- Neste segundo exemplo não será usado o objeto Handler.
- Será utilizado o método "runOnUiThread(Runnable)", que pertence a Activity.
- O objeto Runnable que é passado a este método também é executado dentro da "main thread".

Alteração "MainActivity.java"

```
package usuario.app.threads;

import android.app.Activity;
import android.os.Bundle;
import android.view.View;
import android.view.View.OnClickListener;
import android.widget.Button;
import android.widget.ProgressBar;

public class MainActivity extends Activity {
    private ProgressBar progress;
    private Button startButton;

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        progress = (ProgressBar) findViewById(R.id.progress_bar);
        startButton = (Button) findViewById(R.id.start_button);
    }
}
```

Alteração "MainActivity.java"

```
startButton.setOnClickListener(new OnClickListener() {
    @Override
    public void onClick(View v) {
        Runnable runnable = new Runnable() {
            @Override
            public void run() {
                for (int i = 1; i <= 10; i++) {
                    final int value = i;
                    try {
                        Thread.sleep(1000);
                    } catch (InterruptedException e) {
                        e.printStackTrace();
                    }
                    runOnUiThread(new Runnable() {
                        @Override
                        public void run() {
                            progress.setProgress(value);
                        }
                    });
                }
            }
        });
        new Thread(runnable).start();
    }
});
}
```

- Outra alternativa para utilizar concorrência no Android é utilizar AsyncTasks.
- Um AsyncTask é um objeto que encapsula em uma interface simples o uso de threads.
- Uma AsyncTask deve implementar obrigatoriamente o método `doInBackground()`, que exatamente a tarefa que está sendo executada em background.
- Caso seja necessário alguma atualização na interface, é só sobrescrever o método `onPostExecute()`.
- Tudo que estiver dentro deste método é executado na UI thread.
- Outro método que pode ser sobrescrito é o método `onPreExecute()` que é executado antes do `doInBackground()` e que também é executado na UI thread.